tb_2x1_triang_rew_at_neuron

May 27, 2026

[29]:
```
from __future__ import division
import numpy as np
import matplotlib.pyplot as plt

intp = 10  #tomar uno de cada cien valores, para graficar rapido

# Load data from the .txt file
#ssh -Y cic@148.204.66.123
#ssh -Y alex@148.204.66.53

#source ~/miniconda3/bin/activate
#python3 tb_4x8x4_proc.py
#scp alex@148.204.66.53:'/home/cic/Desktop/EDA/SNN_IPN/sim_results/*.pdf' .
#scp cic@148.204.66.123:'/home/cic/Desktop/EDA/SNN_IPN/sim_results/*.pdf' .

#scp alex@148.204.66.53:/home/alex/.xschem/simulations/data.raw .
#scp alex@148.204.66.53:/home/alex/Desktop/EDA/SNN_IPN/sim_results/
 ↪tb_4x8x4_data.txt .
#scp cic@148.204.66.123:/home/cic/Desktop/EDA/SNN_IPN/sim_results/tb_4x8x4_data.
 ↪txt  .

BSIZE_SP = 512 # Max size of a line of data; we don't want to read the
               # whole file to find a line, in case file does not have
               # expected structure.

MDATA_LIST = [b'title', b'date', b'plotname', b'flags', b'no. variables',
              b'no. points', b'dimensions', b'command', b'option']

def rawread(fname: str):
    """Read ngspice binary raw files. Return tuple of the data, and the
    plot metadata. The dtype of the data contains field names. This is
    not very robust yet, and only supports ngspice.

    >>> darr, mdata = rawread('test.py')
    >>> darr.dtype.names
    >>> plot(np.real(darr['frequency']), np.abs(darr['v(out)']))
    """
```

```
    # Example header of raw file
    # Title: rc band pass example circuit
    # Date: Sun Feb 21 11:29:14  2016
    # Plotname: AC Analysis
    # Flags: complex
    # No. Variables: 3
    # No. Points: 41
    # Variables:
    #         0       frequency       frequency       grid=3
    #         1       v(out)  voltage
    #         2       v(in)   voltage
    # Binary:
    fp = open(fname, 'rb')
    arrs = []
    plots = []
    plot = {}
    while (True):
        try:
            # mdata = fp.readline(BSIZE_SP).split(b':', maxsplit=1)
            mdata = fp.readline().split(b':', maxsplit=1)
        except:
            raise
        if len(mdata) == 2:
            if mdata[0].lower() in MDATA_LIST:
                plot[mdata[0].lower()] = mdata[1].strip()
            if mdata[0].lower() == b'variables':
                nvars = int(plot[b'no. variables'])
                npoints = int(plot[b'no. points'])
                plot['varnames'] = []
                plot['varunits'] = []
                for varn in range(nvars):
                    # varspec = (fp.readline(BSIZE_SP).strip().decode('ascii').
↪split())
                    varspec = (fp.readline().strip().decode('ascii').split())
                    assert(varn == int(varspec[0]))
                    plot['varnames'].append(varspec[1])
                    plot['varunits'].append(varspec[2])
            if mdata[0].lower() == b'binary':
                rowdtype = np.dtype({'names': plot['varnames'],
                                     'formats': [np.complex_ if b'complex'
                                                 in plot[b'flags']
                                                 else np.float64]*nvars})
                # We should have all the metadata by now
                arrs.append(np.fromfile(fp, dtype=rowdtype, count=npoints))
                plots.append(plot)
                plot = {} # reset the plot dict
                fp.readline() # Read to the end of line
```

```
        else:
            break
    return (arrs[0], plots)

data, dicc = rawread("rew_at_neuron.raw")
```

[30]:
```
data
```

[30]:
```
array([(6.25000000e-11,  5.67018130e-05,  1.15353243e+00, 0.01850257,
5.67018999e-05,  1.15353016, 0.01850257, 5.47072849e-05,  0.84627041,
0.01834831,  0.01218788,  0.02326985,  1.10061127e-06, 0.0121879 , 0.02326974,
1.10059749e-06, 0.94483775, 1.63949307, 0.1519008 , 0.8       ),
       (6.42577906e-11,  3.52161118e-06,  1.16907809e+00, 0.01853483,
3.52175591e-06,  1.16907572, 0.01853483, 4.28322347e-06,  0.85740922,
0.01838088,  0.01136734,  0.02273593,  1.12803004e-06, 0.01136736, 0.02273582,
1.12801674e-06, 0.94493574, 1.63906697, 0.15215688, 0.8       ),
       (6.77733717e-11,  1.12887174e-06,  1.19965543e+00, 0.01854627,
1.12900959e-06,  1.19965287, 0.01854628, 1.95998872e-06,  0.87931177,
0.01839322,  0.01068859,  0.02224808,  1.14563925e-06, 0.0106886 , 0.02224796,
1.14562646e-06, 0.94464145, 1.63685819, 0.15223656, 0.8       ),
       …,
       (1.99999389e-02, -2.60820433e-07,  9.46957742e-04, 0.65381114,
5.02617741e-07, -0.00194128, 1.09178671, 1.36633380e-06, -0.00796208,
1.76926838, -0.19331178, -0.18098596,  1.04168800e-06, 0.36387344, 0.3761807 ,
4.00412404e-07, 0.96066865, 0.80322424, 0.46727676, 0.79999999),
       (1.99999694e-02,  4.49606546e-07, -9.39981331e-04, 0.654363  ,
-2.91686550e-07,  0.00194447, 1.09478223, 2.54556774e-06,  0.00808041,
1.76931783, -0.17981202, -0.19201871, -1.03160217e-06, 0.3760463 , 0.36404173,
-3.90562226e-07, 0.9614387 , 0.8032345 , 0.46729257, 0.80000001),
       (2.00000000e-02, -2.62613946e-07,  9.43063671e-04, 0.65501719,
4.99503607e-07, -0.00193852, 1.09772888, 1.35899101e-06, -0.00793482,
1.76926637, -0.19194943, -0.17986045,  1.02163547e-06, 0.36302575, 0.37473464,
3.80939869e-07, 0.96217341, 0.80324441, 0.46727534, 0.80000002)],
      shape=(447022,), dtype=[('time', '<f8'), ('i(v.xx1.vext)', '<f8'),
('v(j1)', '<f8'), ('v(xx1.vm)', '<f8'), ('i(v.xx2.vext)', '<f8'), ('v(j2)',
'<f8'), ('v(xx2.vm)', '<f8'), ('i(v.xx3.vext)', '<f8'), ('v(k1)', '<f8'),
('v(xx3.vm)', '<f8'), ('v(xstdp1.te)', '<f8'), ('v(xstdp1.be)', '<f8'),
('i(v.xstdp1.vmr)', '<f8'), ('v(xstdp2.te)', '<f8'), ('v(xstdp2.be)', '<f8'),
('i(v.xstdp2.vmr)', '<f8'), ('v(vr)', '<f8'), ('v(hx)', '<f8'), ('v(vlk)',
'<f8'), ('v(vin)', '<f8')])
```

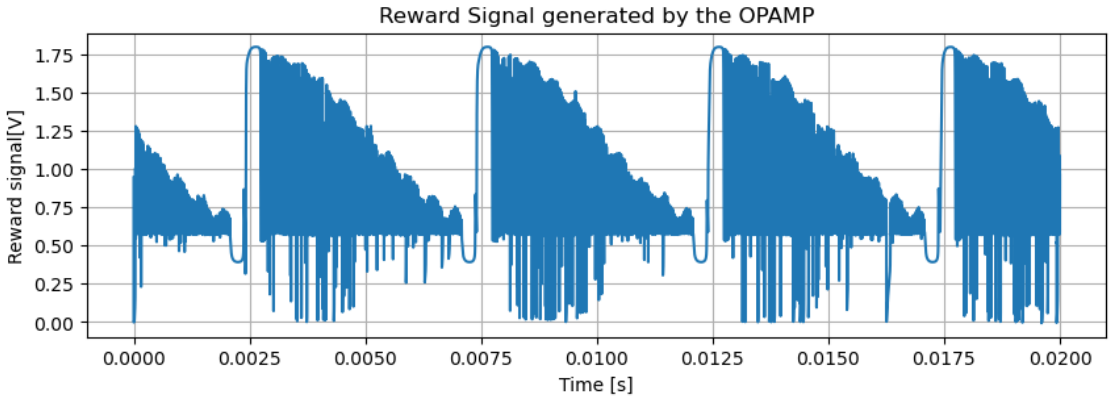
[31]:
```
dicc
```

[31]:
```
[{b'title': b'** sch_path: /foss/designs/snn_ipn/sim_results/triangular_2x1/tb_2
x1_triang_rew_at_neuron.sch',
  b'date': b'Thu May 28 00:32:08  2026',
```

```
b'command': b'ngspice-46, Build Thu Apr 16 19:53:00 UTC 2026',
b'plotname': b'Transient Analysis',
b'flags': b'real',
b'no. variables': b'20',
b'no. points': b'447022',
'varnames': ['time',
 'i(v.xx1.vext)',
 'v(j1)',
 'v(xx1.vm)',
 'i(v.xx2.vext)',
 'v(j2)',
 'v(xx2.vm)',
 'i(v.xx3.vext)',
 'v(k1)',
 'v(xx3.vm)',
 'v(xstdp1.te)',
 'v(xstdp1.be)',
 'i(v.xstdp1.vmr)',
 'v(xstdp2.te)',
 'v(xstdp2.be)',
 'i(v.xstdp2.vmr)',
 'v(vr)',
 'v(hx)',
 'v(vlk)',
 'v(vin)'],
'varunits': ['time',
 'current',
 'voltage',
 'voltage',
 'current',
 'voltage',
 'voltage',
 'current',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'current',
 'voltage',
 'voltage',
 'current',
 'voltage',
 'voltage',
 'voltage',
 'voltage']}]
```

[45]:
```
fig, ax = plt.subplots(1, figsize=(10,3))
ax.plot(data["time"][::intp], data["v(vr)"][::intp], label = "vr")
ax.grid()
ax.set_xlabel("Time [s]")
ax.set_ylabel("Reward signal[V]")
ax.set_title("Reward Signal generated by the OPAMP")
```

[45]: Text(0.5, 1.0, 'Reward Signal generated by the OPAMP')



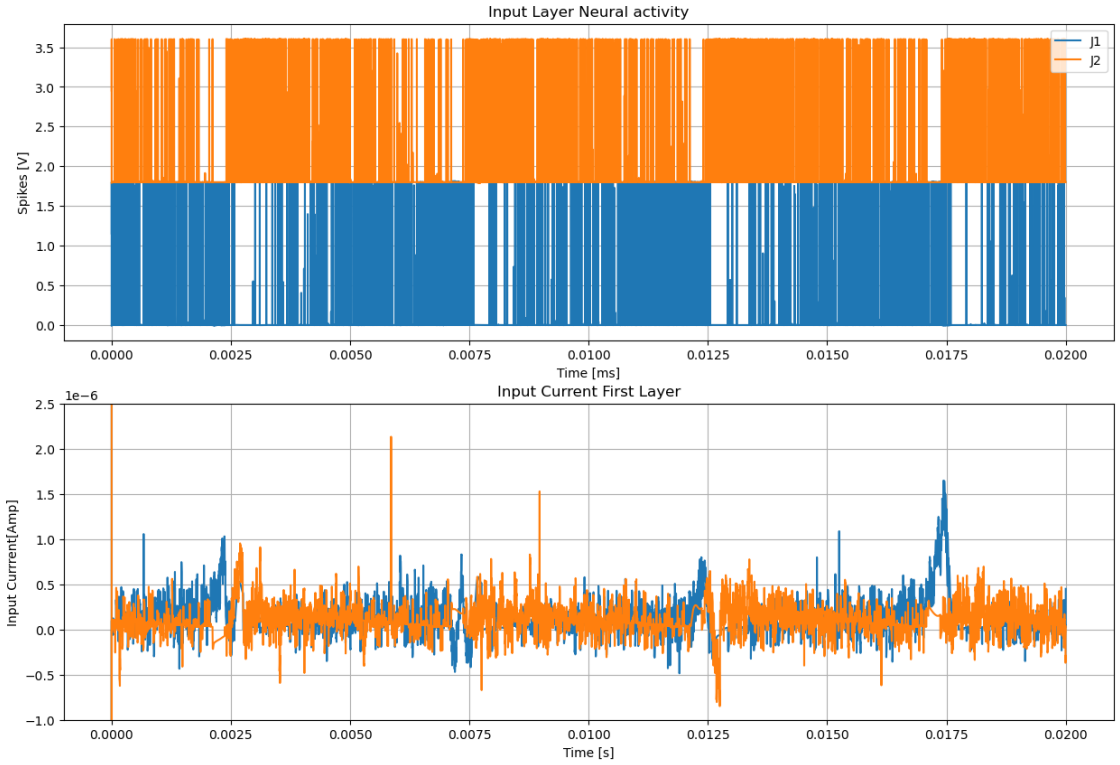
[46]:
```
fig, (ax1, ax2) = plt.subplots(2, figsize=(15,10))

ax1.plot(data["time"][::intp], data["v(j1)"][::intp]+(1.8*0), label = f"J{0+1}")
ax1.plot(data["time"][::intp], data["v(j2)"][::intp]+(1.8*1), label = f"J{1+1}")
ax1.legend()
ax1.grid()
ax1.set_xlabel("Time [ms]")
ax1.set_ylabel("Spikes [V]")
ax1.set_title("Input Layer Neural activity")


ax2.plot(data["time"][::intp], data["i(v.xx1.vext)"][::intp], label = f"i{0+1}")
ax2.plot(data["time"][::intp], data["i(v.xx2.vext)"][::intp], label = f"i{0+1}")
ax2.grid()
ax2.set_xlabel("Time [s]")
ax2.set_ylabel("Input Currrent[Amp]")
ax2.set_title("Input Current First Layer")
ax2.set_ylim((-0.1e-5, 0.25e-5))

# fig.savefig('InputLayer.pdf')
```

[46]: (-1e-06, 2.5e-06)



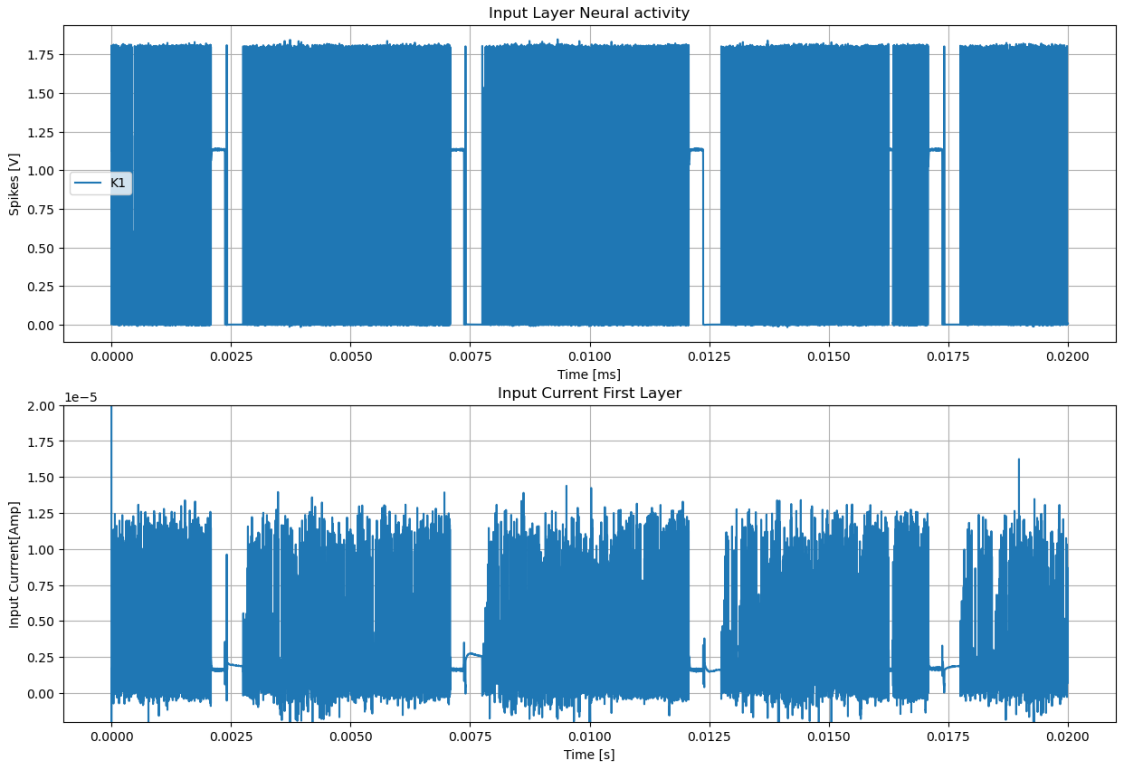
[51]:
```
fig, (ax1, ax2) = plt.subplots(2, figsize=(15,10))

ax1.plot(data["time"][::intp], data["v(k1)"][::intp], label = "K1")
ax1.legend()
ax1.grid()
ax1.set_xlabel("Time [ms]")
ax1.set_ylabel("Spikes [V]")
ax1.set_title("Input Layer Neural activity")


ax2.plot(data["time"][::intp], data["i(v.xx3.vext)"][::intp], label = f"i{0+1}")
ax2.grid()
ax2.set_xlabel("Time [s]")
ax2.set_ylabel("Input Currrent[Amp]")
ax2.set_title("Input Current First Layer")
ax2.set_ylim((-0.2e-5, 2e-5))

# fig.savefig('InputLayer.pdf')
```

[51]: (-2e-06, 2e-05)



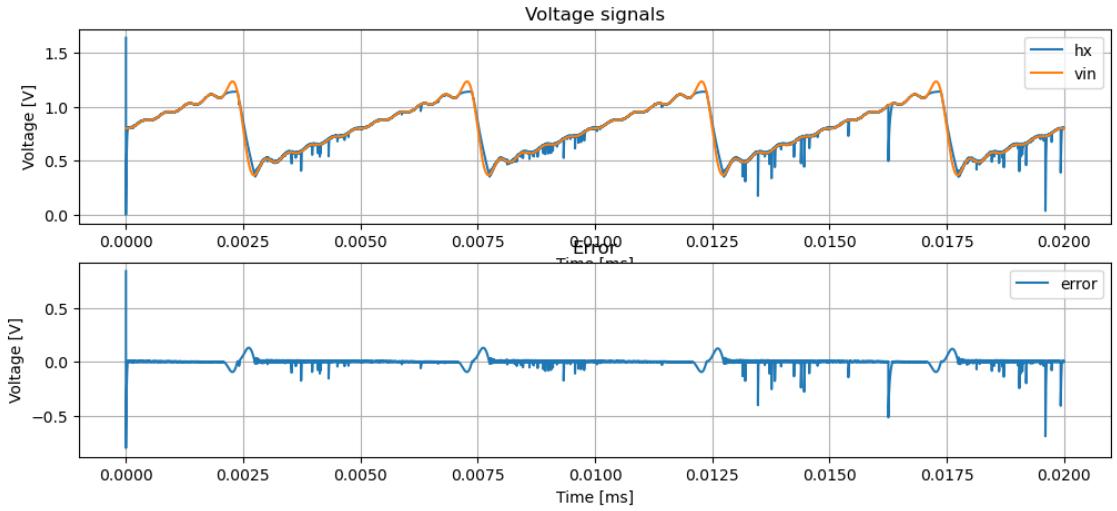
[35]:
```
fig, [ax1, ax2] = plt.subplots(2, figsize=(12,5))

ax1.plot(data["time"][::intp], data["v(hx)"][::intp], label = "hx")
# ax1.plot(data["time"][::intp], data["v(x)"][::intp], label = "x")
ax1.plot(data["time"][::intp], data["v(vin)"][::intp], label = "vin")
ax1.legend()
ax1.grid()
ax1.set_xlabel("Time [ms]")
ax1.set_ylabel("Voltage [V]")
ax1.set_title("Voltage signals")


ax2.plot(data["time"][::intp], data["v(hx)"][::intp] - data["v(vin)"][::intp],␣
 ↪label = "error")
ax2.legend()
ax2.grid()
ax2.set_xlabel("Time [ms]")
ax2.set_ylabel("Voltage [V]")
ax2.set_title("Error")


# fig.savefig('Signals.pdf')
```

[35]: Text(0.5, 1.0, 'Error')

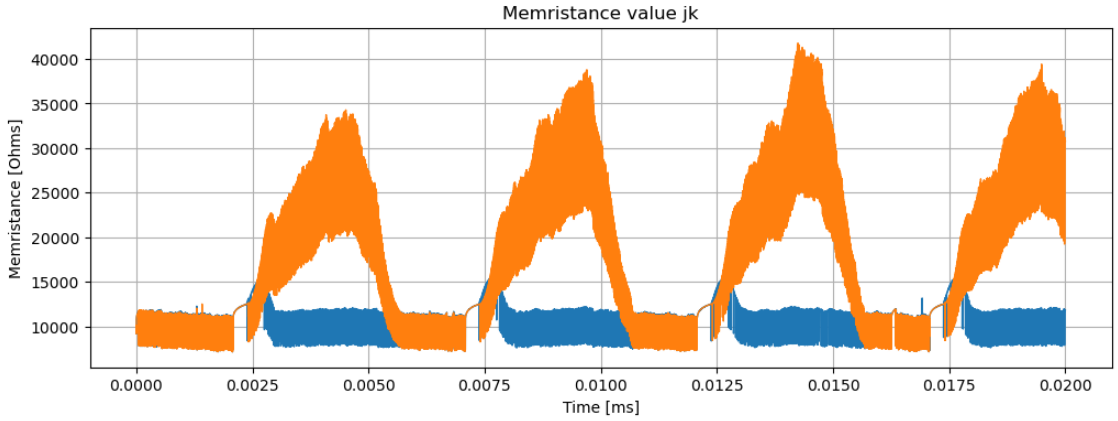


[39]:
```
fig, ax2 = plt.subplots(1, figsize=(12,4))

mem_jk = []
for k in range(2):
    Imem_label = "i(v.xstdp" +  str(k+1) + ".vmr)"
    Vte_label =  "v(xstdp" +  str(k+1) + ".te)"
    Vbe_label = "v(xstdp" +  str(k+1) + ".be)"
    Imem= data[Imem_label][::intp]
    Vte = data[Vte_label][::intp]
    Vbe = data[Vbe_label][::intp]
    mem = (Vbe - Vte)/Imem
    ax2.plot(data["time"][::intp], mem, label = f"jk{k+1}",linewidth=1,␣
 ↪markersize=2)
    mem_jk.append(mem)


# ax.legend()
ax2.grid()
ax2.set_xlabel("Time [ms]")
ax2.set_ylabel("Memristance [Ohms]")
ax2.set_title("Memristance value jk")
# ax2.set_ylim((-0.2e6, 3.5e6))
# fig.savefig('weights.pdf')
```

[39]: Text(0.5, 1.0, 'Memristance value jk')


Memristance value jk
Memristance [Ohms]
40000
35000
30000
25000
20000
15000
10000
0.0000
0.0025
0.0050
0.0075
0.0100
0.0125
0.0150
0.0175
0.0200
Time [ms]

[ ]: